Citation-Aware Retrieval-Augmented Generation for Document-Grounded Question Answering

Veenus K K
M.Tech Machine Learning
Indian Institute of Space Science and Technology (IIST), Trivandrum
veenuskk02@gmail.com

Abstract

Retrieval-Augmented Generation (RAG) systems improve factual grounding by combining external document retrieval with language generation models. However, hallucination and lack of source transparency remain important challenges in practical question answering systems. This work presents a citation-aware Retrieval-Augmented Generation pipeline for document-grounded machine learning question answering using transformer embeddings, FAISS-based semantic retrieval, and lightweight pretrained language models. The proposed system extracts text from machine learning documents, performs chunking with overlap preservation, generates semantic embeddings using the all-MiniLM-L6-v2 embedding model, and retrieves semantically relevant chunks using cosine similarity search through FAISS. Citation-aware prompting is incorporated to improve explainability and source traceability in generated responses. Multiple experiments were conducted to analyze the effect of retrieval parameters such as top-k chunk selection and similarity metrics(L2 distance and cosine similarity). Experimental observations show that cosine similarity with normalized embeddings provides semantically meaningful retrieval behavior, while citation-aware prompting improves interpretability and trustworthiness of generated answers.

1 Introduction

Large Language Models (LLMs) have shown strong capabilities in natural language understanding and generation. However, these models often suffer from hallucination, where generated responses contain unsupported or factually incorrect information.

Retrieval-Augmented Generation (RAG) addresses this issue by combining external document retrieval with language generation. Instead of relying entirely on the internal knowledge of a language model, RAG systems retrieve relevant contextual information from external documents before answer generation.

This project presents a citation-aware Retrieval-Augmented Generation pipeline for document-grounded machine learning question answering. The system combines transformer-based semantic embeddings, FAISS vector retrieval, and prompt-engineered language generation to produce grounded responses with citations.

The major contributions of this work are:

- Development of a citation-aware RAG pipeline for document-grounded question answering
- Implementation of semantic retrieval using transformer embeddings and FAISS
- Experimental analysis of top-k retrieval settings
- Comparative analysis of cosine similarity and Euclidean distance retrieval

Preprint. Under review.

- Study of citation-aware prompting for improving explainability and trustworthiness

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines semantic retrieval systems with language generation models. The retrieval module identifies relevant document chunks corresponding to a user query, while the language model generates answers conditioned on the retrieved context.

The general RAG workflow consists of:

1. User query encoding
2. Semantic retrieval
3. Context construction
4. Prompt generation
5. Answer generation

2.2 Semantic Embeddings

Semantic embeddings are dense vector representations that capture semantic meaning of text. Transformer-based embedding models map semantically similar sentences closer in vector space.

This project uses the sentence-transformer model:

all-MiniLM-L6-v2

2.3 Vector Similarity Search

Semantic retrieval requires efficient nearest-neighbor search in high-dimensional embedding spaces. FAISS (Facebook AI Similarity Search) provides scalable similarity search for vector embeddings.

The project uses:

- FAISS IndexFlatIP
- Cosine similarity retrieval with normalized embeddings

3 System Architecture

The complete system workflow is:

PDF Documents $\rightarrow$ Text Extraction $\rightarrow$ Chunking $\rightarrow$ Embedding Generation $\rightarrow$ FAISS Indexing $\rightarrow$ Query Embedding $\rightarrow$ Top-k Retrieval $\rightarrow$ Prompt Construction $\rightarrow$ LLM Generation

PDF Documents

- Wikipedia PDFs
- GeeksforGeeks PDFs
- ML Notes

$\downarrow$

Text Extraction & Chunking

- Extract text
- Create chunks
- Store metadata (.json file)
- store citation information
- store metadata and citation info as dict

↓

Embedding Generation

- all-MiniLM-L6-v2
- Vector embeddings
- Semantic representation

↓

FAISS Vector Database

- Store embeddings
- Similarity search (cosine similarity)
- Retrieve top-k chunks

User Query

Question Embedding

↓

Top-k Retrieval

↓

Large Language Model

- Phi-2 (used)/ DistilGPT2
- Context-grounded answer
- Citation generation

↓

Final Output

Answer + Citations

Example:

- Generated Answer
-

$doc\_03, Page2$

4 Methodology

4.1 Dataset Collection

The dataset consists of 12 machine learning related PDF documents collected from educational and technical sources including:

- Wikipedia
- GeeksforGeeks
- Machine learning lecture notes

The selected documents contain machine learning concepts such as:

- Machine Learning
- Linear Regression
- Logistic Regression
- Cost function
- Gradient Descent
- Backpropagation
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Regularization
- Neural Networks
- Principal Component Analysis (PCA)
- Overfitting

4.2 Text Extraction and Chunking

PDF text extraction was performed using PyMuPDF. The extracted text was cleaned and divided into overlapping chunks.

4.2.1 Chunking Configuration

- Chunk size = 400 words
- Overlap = 50 words

Chunk overlap preserves contextual continuity across chunk boundaries.

4.3 Embedding Generation

Semantic embeddings were generated using:

all-MiniLM-L6-v2

Embeddings were generated using:

`normalize_embeddings=True`

The cosine similarity between vectors is defined as:

$$\cos(\theta) = \frac{x \cdot y}{||x||||y||} \tag{1}$$

After normalization:

$$||x|| = ||y|| = 1 \tag{2}$$

Thus:

$$\cos(\theta) = x \cdot y \tag{3}$$

4.4 FAISS Vector Indexing

The generated embeddings were stored using FAISS.

The following FAISS index was used:

```
faiss.IndexFlatIP
```

This configuration performs inner-product similarity search, which effectively becomes cosine similarity retrieval for normalized embeddings.

4.5 Query Retrieval Pipeline

The retrieval pipeline consists of:

1. User query input
2. Query embedding generation
3. Semantic retrieval using FAISS
4. Top-k chunk selection
5. Context construction

4.6 Citation-Aware Prompting

Citation-aware prompting was introduced to improve explainability and source grounding.

Retrieved chunks were labeled using identifiers such as:

```
Source [1]
Source [2]
```

The prompt instructed the language model to:

- answer only using retrieved context
- include citations
- avoid external knowledge

4.7 Language Model Integration

Initially, the following lightweight language model was used:

distilgpt2

Later experiments explored:

microsoft/phi-2

The project focuses on inference and retrieval analysis rather than model training.

5 Experimental Setup

5.1 Hardware Configuration

Experiments were conducted on:

- Apple MacBook Air M3
- Apple Silicon architecture
- MPS GPU acceleration support

5.2 Software Environment

The implementation used:

- Python
- PyTorch
- sentence-transformers
- HuggingFace transformers
- FAISS
- PyMuPDF

5.3 Evaluation Questions

The evaluation set included machine learning related conceptual questions:

- What is machine learning?
- Explain linear regression.
- Explain gradient descent in machine learning.
- What is backpropagation?
- Define Support Vector Machine.
- Explain regularization.
- Explain KNN.
- What is overfitting?
- What are neural networks?
- Explain PCA dimensionality reduction.

6 Experiments and Results

6.1 Experiment 1: Effect of Top-k Retrieval

6.1.1 Objective

To analyze how the number of retrieved chunks influences contextual quality and answer generation.

6.1.2 Configurations

- k = 2
- k = 5
- k = 10

6.1.3 Observations

Table 1: Effect of top-k retrieval

Top-k	Observation
2	Insufficient contextual information
5	Best balance between relevance and completeness
10	Increased retrieval noise

When k was too small, important contextual information was missing from the retrieved chunks, resulting in incomplete answers. Increasing k improved contextual coverage but also introduced irrelevant chunks.

6.2 Experiment 2: Cosine Similarity vs Euclidean Distance

6.2.1 Objective

To compare semantic retrieval performance between:

- IndexFlatL2
- IndexFlatIP with normalized embeddings

6.2.2 Observation

Both methods produced highly similar retrieval results for most evaluation queries.

6.2.3 Mathematical Explanation

For normalized embeddings:

$$||x - y||^2 = 2 - 2(x \cdot y) \tag{4}$$

This relationship explains why cosine similarity and Euclidean distance often preserve similar nearest-neighbor rankings after embedding normalization.

6.3 Experiment 3: Citation-Aware Prompting

6.3.1 Objective

To study the effect of citation-aware prompting on generated responses.

6.3.2 Observation

Citation-aware responses improved:

- explainability
- source traceability
- interpretability
- user trust

The generated responses became more grounded to retrieved documents.

7 Discussion

The experiments demonstrate that retrieval quality significantly affects grounded language generation.

The following observations were particularly important:

- Retrieval quality strongly influences generated answer relevance
- Retrieval noise increases with excessively large top-k values
- Embedding normalization influences retrieval behavior
- Cosine similarity provides semantically meaningful retrieval
- Citation-aware prompting improves explainability

The work highlights that effective RAG systems depend not only on language models but also on retrieval quality, chunking strategy, and prompt engineering.

8 Limitations

The project has several limitations:

- The dataset size is relatively small
- Lightweight pretrained language models were used due to hardware constraints
- Citation formatting was not always perfectly followed
- Evaluation was primarily qualitative
- No reranking stage was implemented

9 Conclusion

This work presented a citation-aware Retrieval-Augmented Generation pipeline for document-grounded machine learning question answering.

The system combined:

- transformer-based semantic embeddings
- FAISS vector retrieval
- citation-aware prompting
- pretrained language models

The results showed that:

- top-k retrieval significantly influences contextual quality
- cosine similarity with normalized embeddings provides semantically meaningful retrieval
- citation-aware prompting improves explainability and source traceability

Future work may include:

- reranking techniques
- larger instruction-tuned language models
- automated citation verification
- quantitative hallucination metrics

References

1. Lewis, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.
item Johnson, Jeff, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs." IEEE Transactions on Big Data, 2019.
2. Reimers, Nils and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP, 2019.
item Vaswani, Ashish et al. Attention Is All You Need." NeurIPS, 2017.
3. Devlin, Jacob et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL, 2019.

A Appendix

A.1 Sample Retrieval Pipeline

```
query_embedding = model.encode([query], normalize_embeddings=True)

k = 5

results = index.search(query_embedding, k)
```

A.2 Sample Citation-Aware Prompt

```
Answer the question ONLY using the provided context.

Rules:

1. Every statement must include citation.
2. Do not use outside knowledge.
3. If answer is not found, say ‘‘Not found in context’’.
```